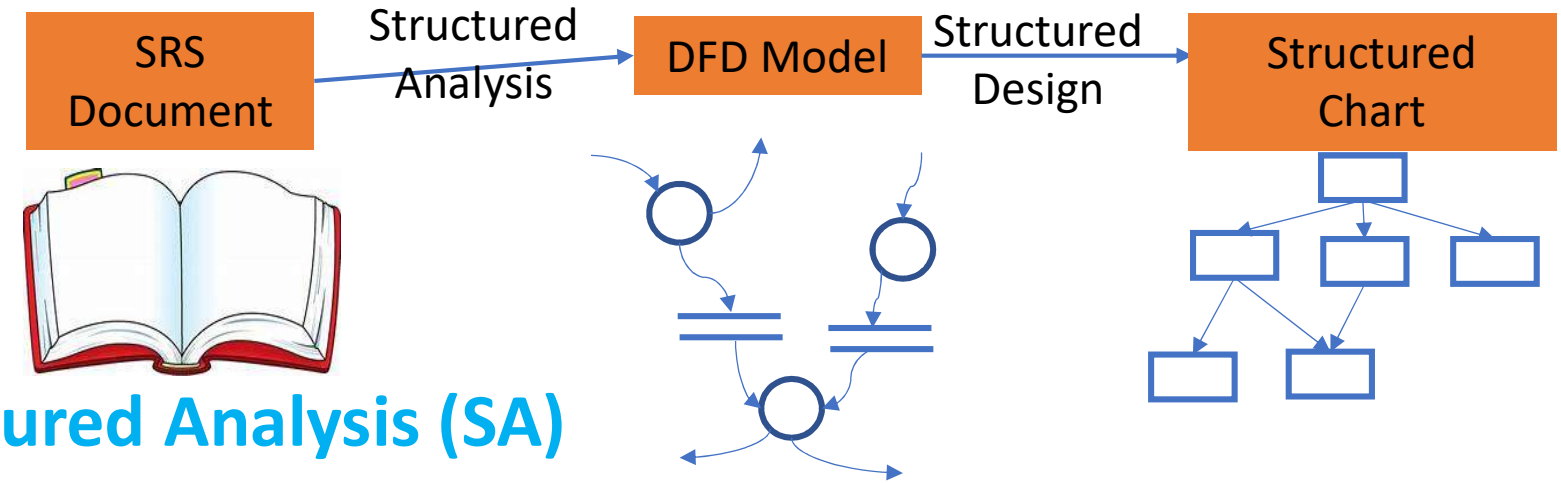
Function Oriented Software Design

S. S. Satapathy

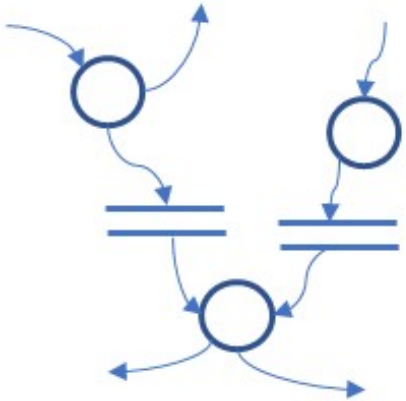
SA/SD Methodology



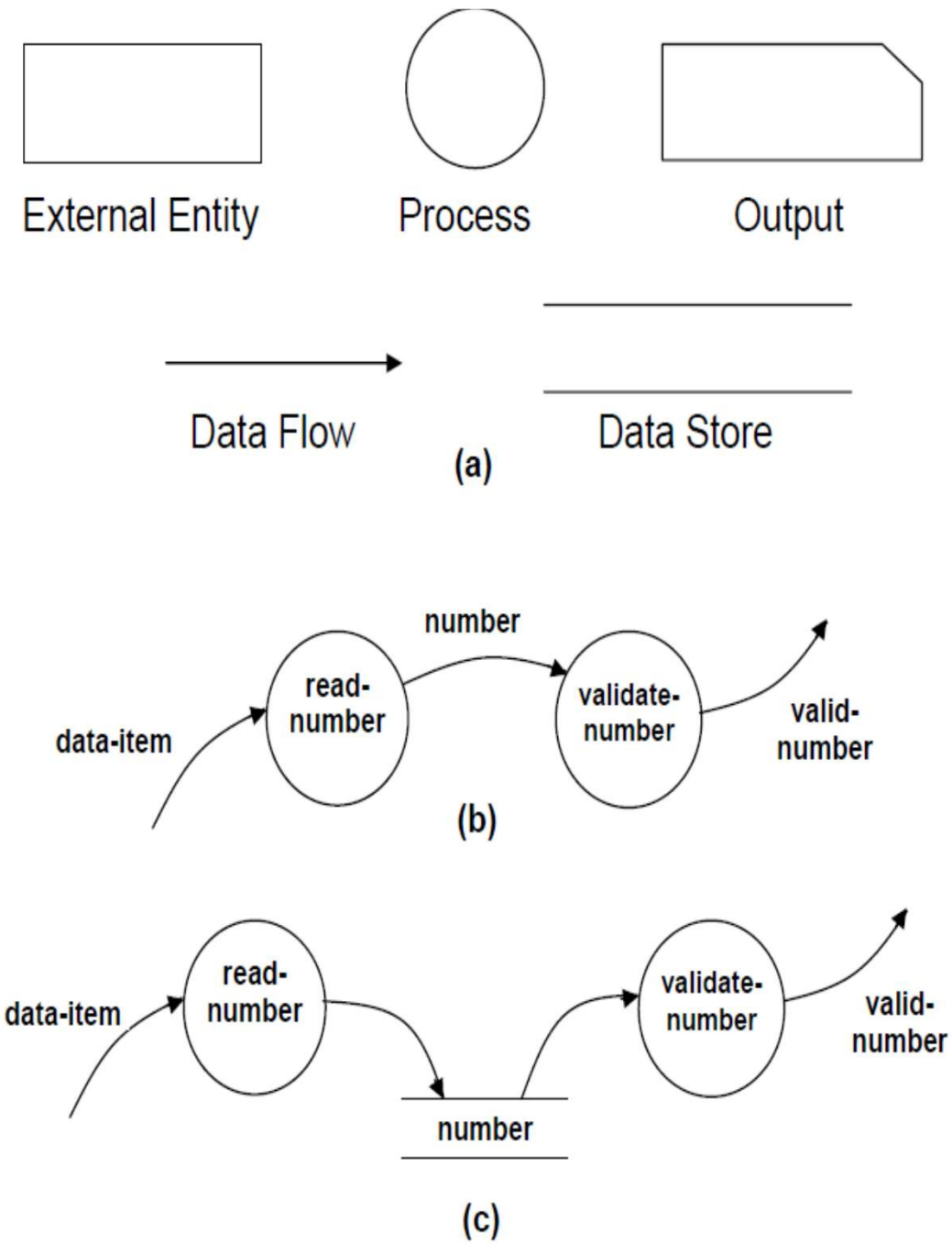
- **Step-I: Structured Analysis (SA)**
  - **Transforms SRS document into a graphical model called DFD (data flow diagram) models**
  - That is, each function the system need to perform is analyzed and hierarchically decomposed in to more detailed functions.
- **Step-II: Structured Design (SD)**
  - **All functions identified are mapped into a module structure**. This module structure is known as **high level design** or **software architecture**.
- **SA is to capture detailed structure of the system as perceived by the user**, whereas the **purpose of SD is to define structure of the system that is suitable for implementation**.

Structured Analysis

- **SA: A top-down decomposition approach**
- Application of **divide and conquer technique**. Each **high level function is decomposed in to detailed functions**.
- **Graphical representation of the analysis results using Data Flow Diagrams (DFD)**.
  - **DFD**: A **hierarchical graphical model of a system** that **shows the different processing activities or functions that the system performs and the data interchange among those functions**.
  - **DFD** is also known as **bubble chart**.



Data Flow Diagrams (DFDs)



(a) Symbols used for designing DFDs
(b) Synchronous data flow
(c) Asynchronous data flow

Data Dictionary

- **A data dictionary lists all data items appearing in the DFD model of a system**.
- The **data items listed include all data flows and the contents of all data stores** appearing on the DFDs in the DFD model of a system.
- A **data dictionary lists the purpose of all data items** and the **definition of all composite data items in terms of their component data items**.

Data Definition Operators

- **+: denotes composition** of two data items, e.g. a+b represents data a and b.
- **[,,]: represents selection**, i.e. any one of the data items listed in the brackets can occur. For example, [a,b] represents either a occurs or b occurs.
- **()**: the contents inside the bracket **represent optional data** which may or may not appear. e.g. a+(b) represents either a occurs or a+b occurs.
- **{}: represents iterative data definition**, e.g. {name}5 represents five name data. {name}* represents zero or more instances of name data.
- **=: represents equivalence**, e.g. a=b+c means that a represents b and c.
- **/* */:** Anything appearing within /* and */ is considered as **a comment**.

Developing a DFD Model of a System

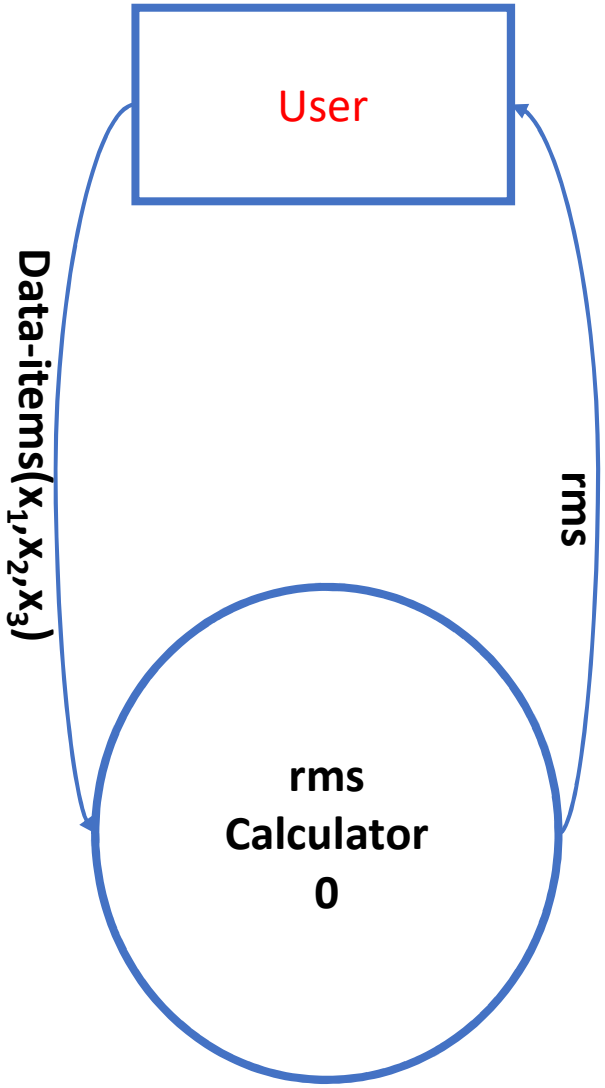
The DFD model consists of hierarchy of many DFDs and a single data dictionary

- **The top level DFD** is called as **level 0 DFD** or **context diagram**. This is the **most abstract or simplest representation of the system (highest level)**.
- **Context diagram establishes the context in which the system operates**, i.e. who are the users, what data they input to the system, what data they receive from the system.

DFD Example 1
RMS Calculating Software

- A software system called RMS calculating software would read three integral numbers from the user in the range of -1000 and +1000 and then determine the root mean square (rms) of the three input numbers (x_1, x_2, x_3) and display it.
- rms(x_1, x_2, x_3): $= \sqrt{(x_1^2 + x_2^2 + x_3^2)/3}$

- **The system accepts three integers from the user and returns back the result.**

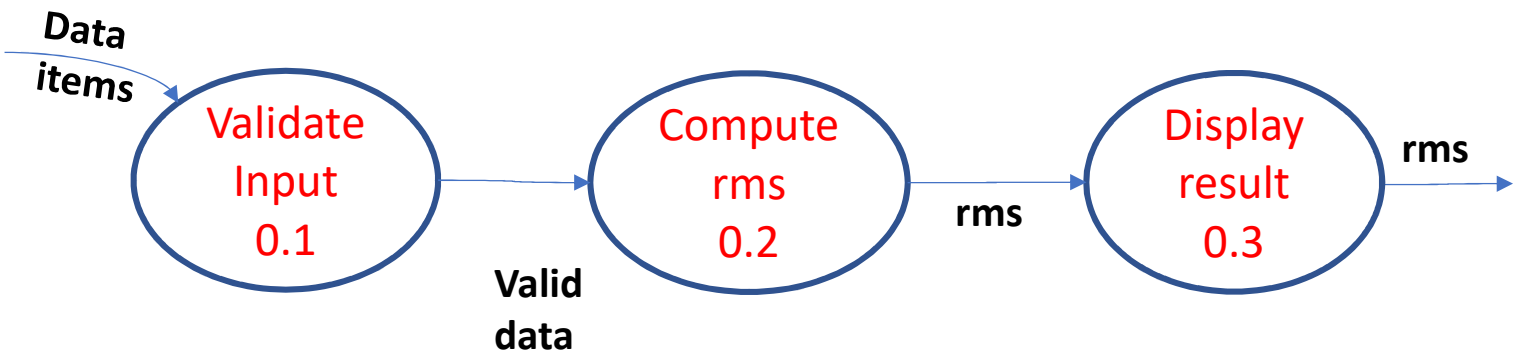


DFD level 0: Context Diagram

Developing a DFD contd...

- At each successive layer, more details are gradually introduced.
- Usually, level 0 and 1 consists of only one DFD, and subsequent levels consists of more DFDs.
- When a **bubble numbered x** is **decomposed**, its **children bubbles** are **numbered x.1, x.2, x.3, etc**. In this numbering scheme, by looking at the number of a bubble we can unambiguously determine its level, its ancestors, and its successors.

RMS Calculating Software



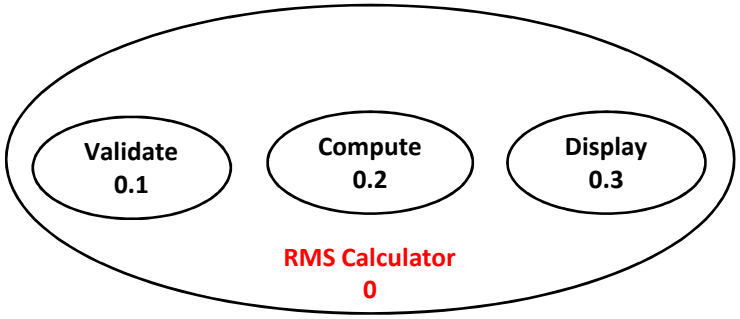
DFD level 1

Data dictionary for the DFD model

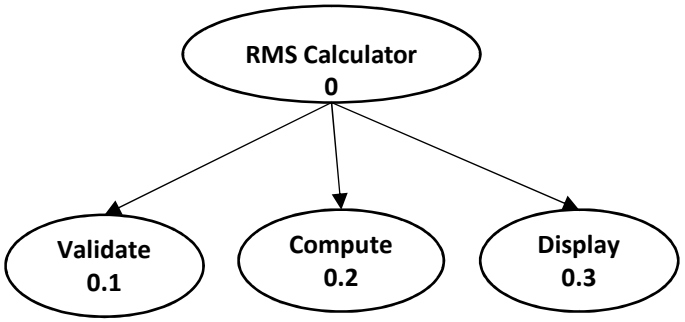
- Data-items: $x_1+x_2+x_3$
- x_1: float
- x_2: float
- x_3: float
- rms: float

Important points on DFD

- **Level 1 DFD**
  - Usually it **consists of 3 to 7 bubbles**. Each **bubble represents a function performed by the system**. That means **system is representing 3 to 7 important functions**.
- Usually **in any system**, there can be **up to 3rd or 4th level of DFDs**
- **Decomposition**
  - **Each bubble of upper level further decomposed into several sub-functions in lower level**.



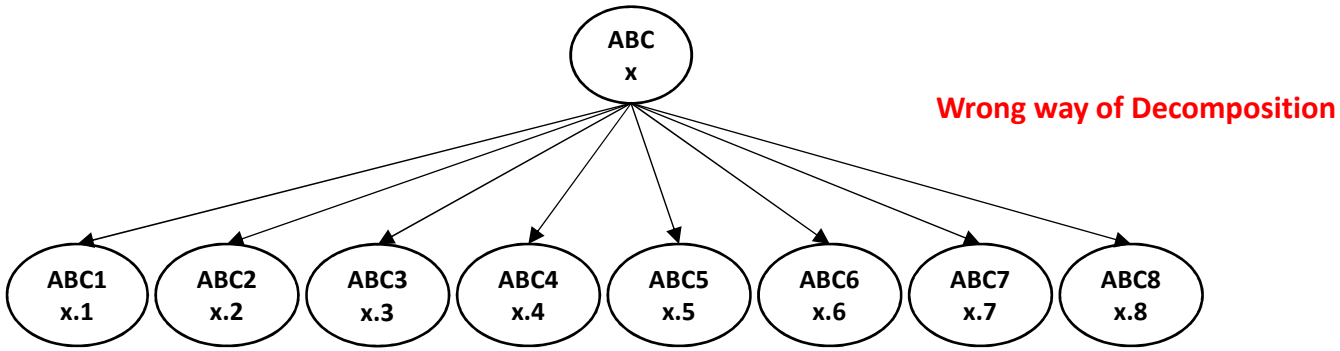
Bubble of a level is decomposed in to 3 bubbles in next level



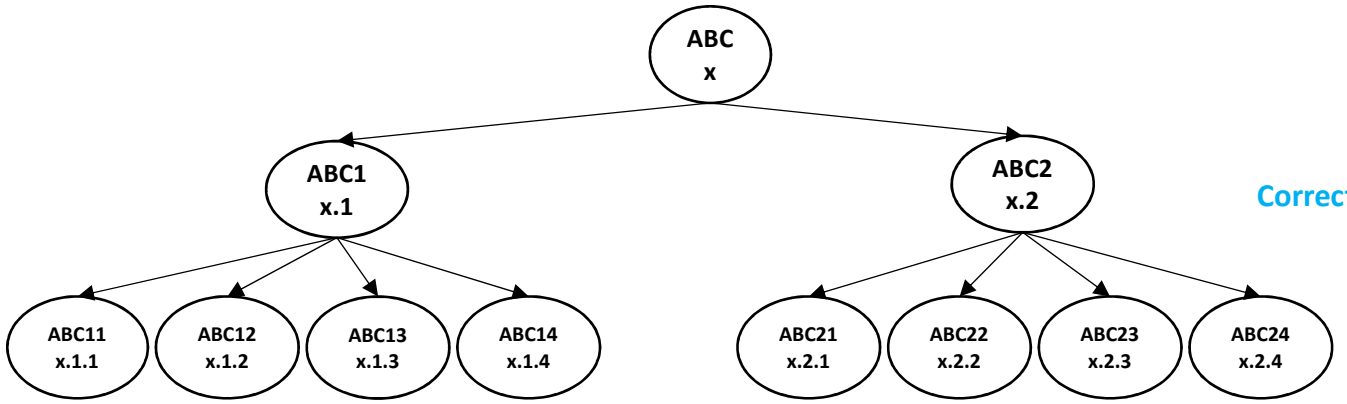
Two level DFD

Important points on DFD

- **Decomposition**
  - In case a **bubble is to be decomposed** in to **more than 7 bubbles**, then instead of **representing all the bubbles** in **one level**, **represent** them in **two levels** as shown in figure below.



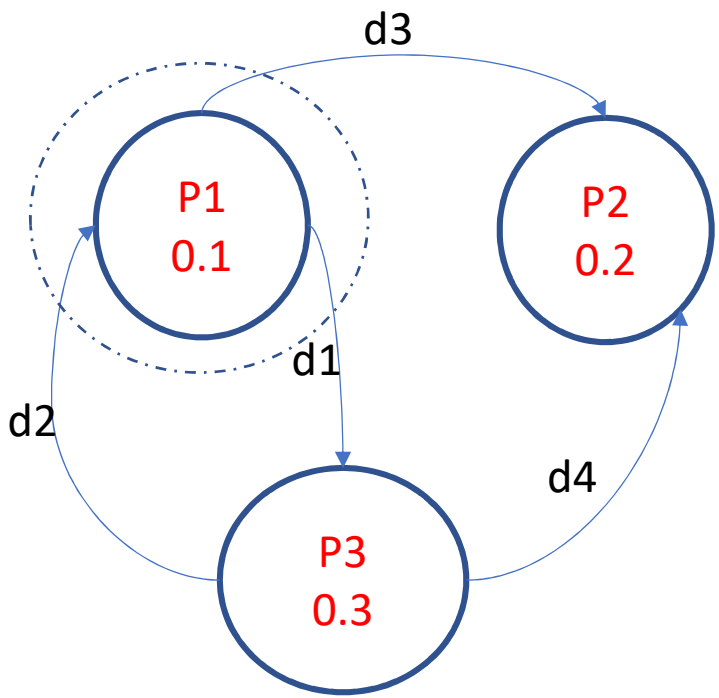
Wrong way of Decomposition



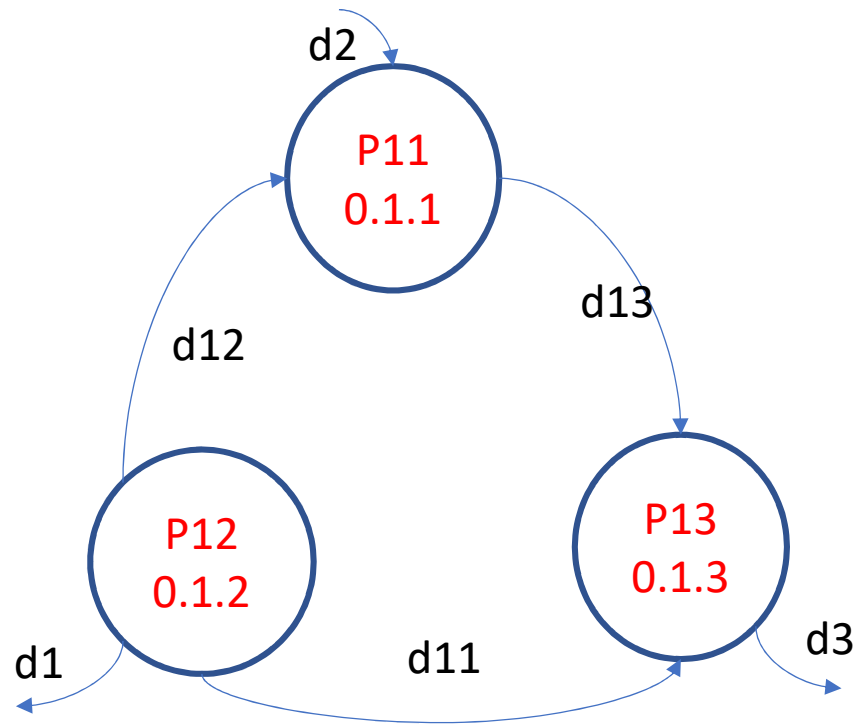
Correct way of Decomposition

Balancing of a DFD

- The **data** that **flow into** or **out of a bubble** in a **DFD must match** the **data flow** at the **next level** of **DFD**.



Level -1 DFD

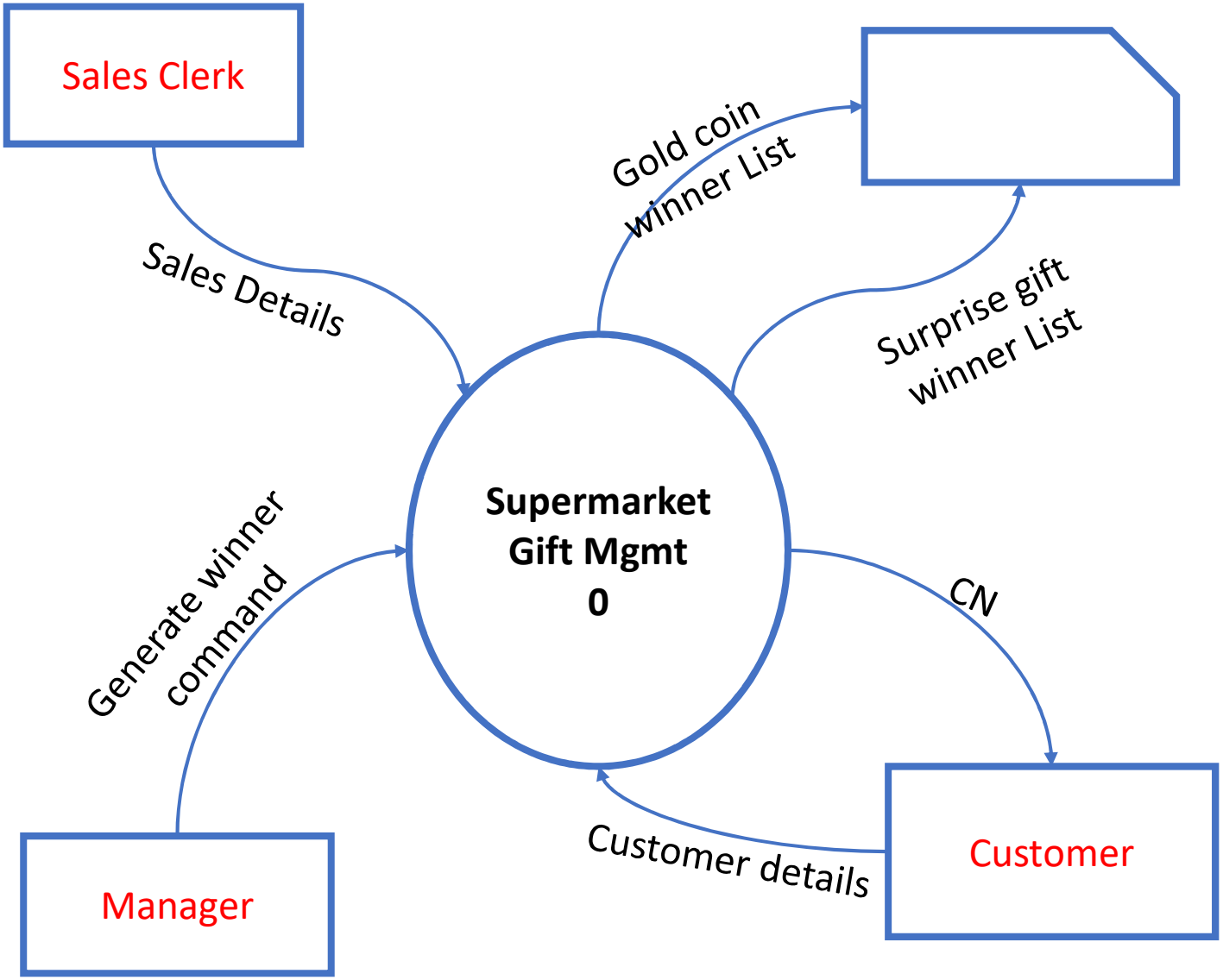


Level -2 DFD

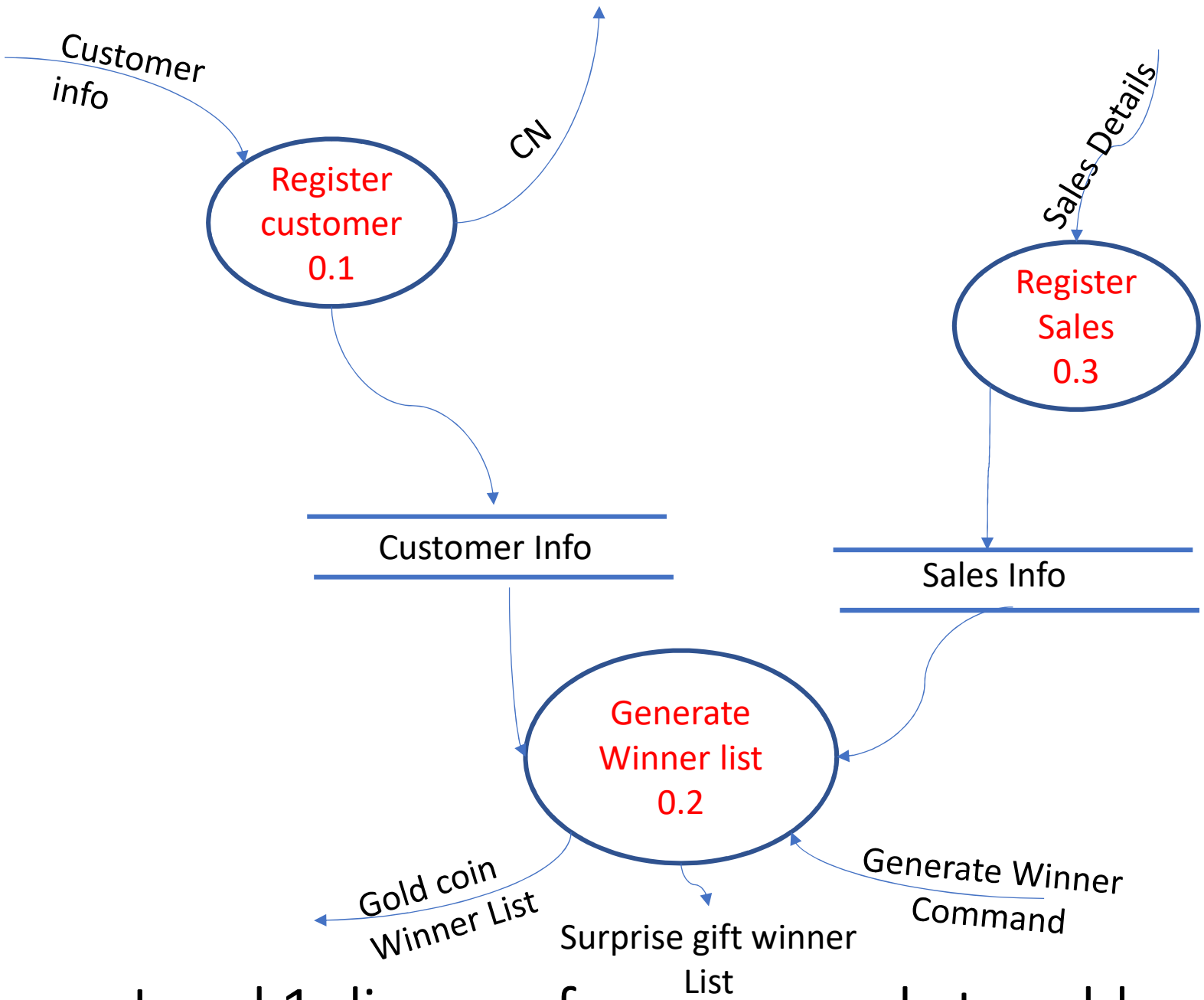
An example showing balanced decomposition

Supermarket Gift Mgmt example

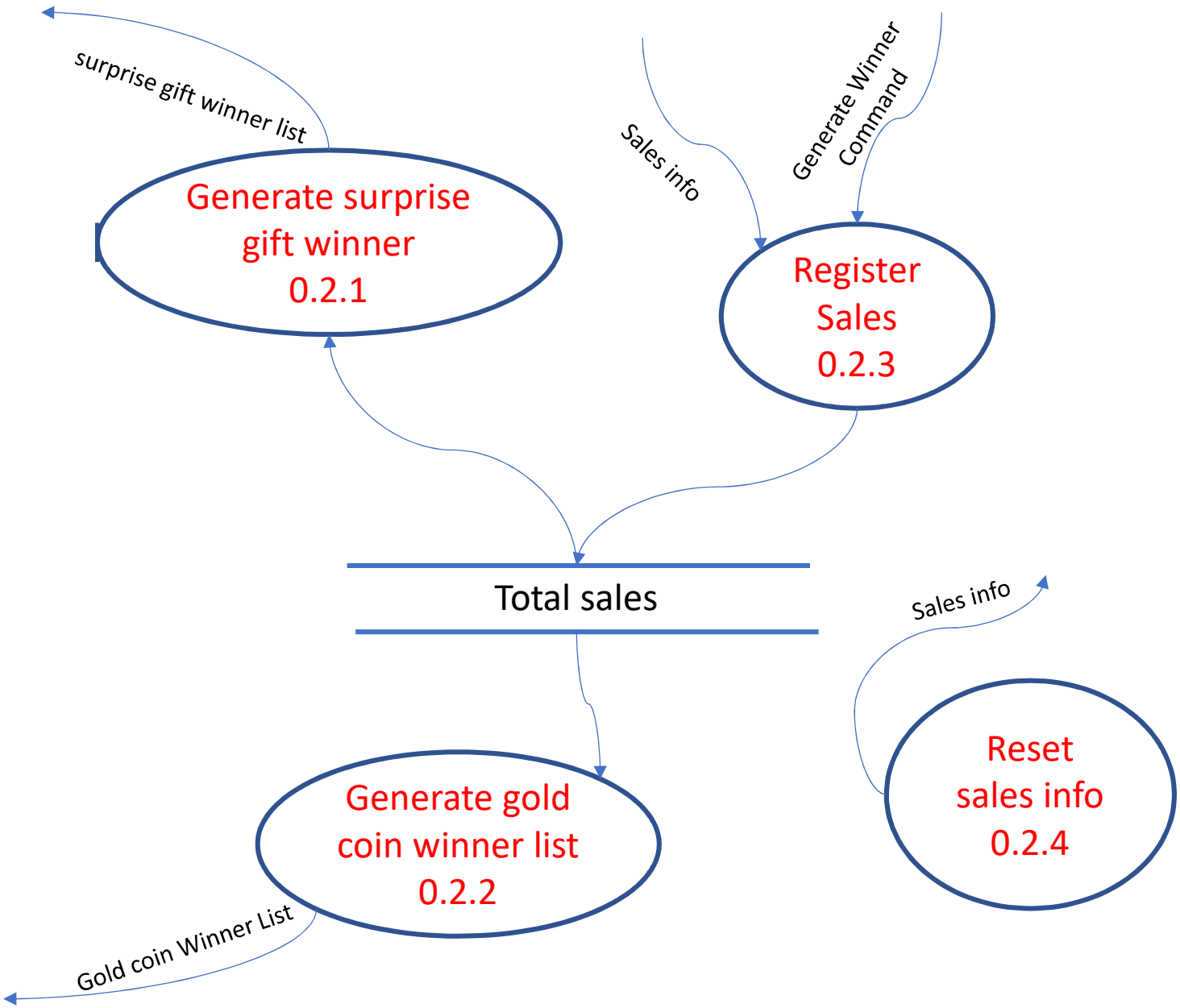
- **Customer** needs to **register in the supermarket by giving his/her details** such as residence address, telephone number, and the driving license number.
- **Each customer** who registers for this scheme is **assigned a unique customer number (CN)** by the computer.
- A **customer** can **present his/her CN** to the check out staff when **he/she makes any purchase**. In this case, the value of his/her **purchase is credited against his/her CN**.
- **At the end of each year**, the **supermarket awards surprise gifts** to **10 customers who make the highest total purchase** over the year.
- Also, **awards a 22 caret gold coin** to every customer **whose purchase exceeded Rs.10,000**.
- The **entries against the CN are reset** on the day of every year **after the prize winners' lists are generated**.



Level 0 DFD for supermarket problem



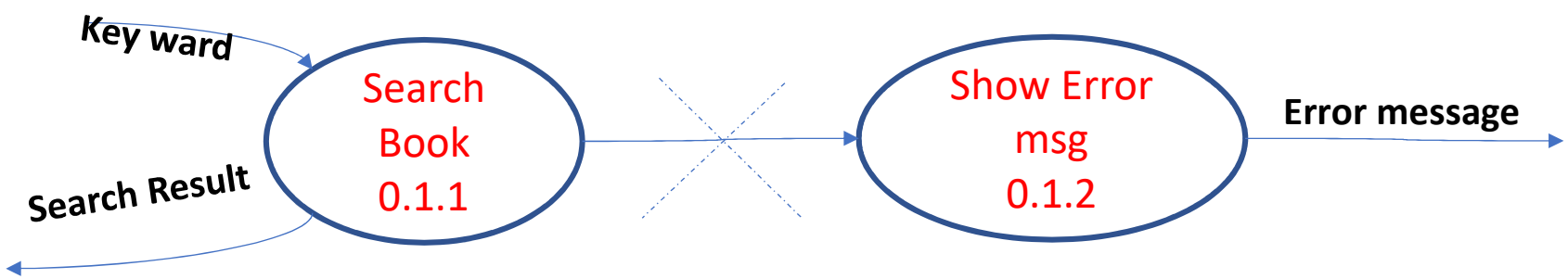
Level 1 diagram for supermarket problem



Level 2 diagram for supermarket problem

Errors in DFD construction

- **No more than one bubble is allowed in the context diagram**. A context diagram should depict the system as a single bubble.
- All **external entities interacting with the system should be represented only in the context diagram**. The **external entities should not appear at other levels of the DFD**.
- **Too less or too many bubbles are not allowed in a DFD**. Only 3 to 7 bubbles per diagram should be allowed, i.e. each bubble should be decomposed to between 3 and 7 bubbles.
- **No DFD should be left unbalanced**
- **Control information are not represented in a DFD**



Errors in DFD Construction

- **A data store should be connected only to bubbles through data arrows**. A data store cannot be connected to another data store or to an external entity.
- **All the functionalities of the system must be captured by the DFD model**. No function of the system specified in its SRS document should be overlooked.
- **Only those functions of the system specified in the SRS document should be represented**, i.e. the **designer should not assume functionality of the system not specified by the SRS document** and then try to represent them in the DFD.
- **Improper or unsatisfactory data dictionary not allowed**.
- **The data and function names must be intuitive**, **symbolic data names such a, b, c, etc should not be used**.

Importance of DFDs

- **Starting with a set of high-level functions** that a system performs, a **DFD model hierarchically represents** various **sub-functions**.
- **Starting with** a very simple and **abstract model** of a system, different **details of the system are slowly introduced** through different **hierarchies**.
- Along with representation of the results of structured analysis, **DFD also shows the flow of documents or items in an organization**.

Importance of data dictionary

- A **data dictionary provides a standard terminology for all relevant data** for use by the engineers working in a project.
- It is very **important to avoid confusion in large projects** by avoiding use of different terms to refer to the same data
- The **data dictionary** provides the analyst with a means to determine the **definition of different data structures** in terms of their component elements.

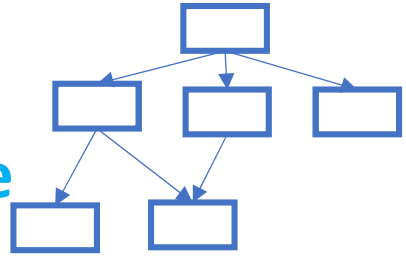
Shortcomings of a DFD model

- **DFDs leave ample scope to be imprecise**
  - In the DFD model, the **function performed by a bubble is judged from its label**. However, a **short label** **may not capture the entire functionality of a bubble**.
- **Control aspects are not defined by a DFD**
  - For instance, the **order in which inputs are consumed and outputs are produced** by a bubble is **not specified**.
  - **A DFD model does not specify the order in which the different bubbles are executed**. Representation of such aspects is very important for modeling real-time systems.
- **The method of carrying out decomposition** to arrive at the successive levels and the ultimate level to which decomposition is carried out **are highly subjective** and **depend** on the choice and **judgment of the analyst**.
  - Due to this reason, **even for the same problem, several alternative DFD representations are possible**. And it is difficult to decide which is better
- The **data flow diagramming technique does not provide any specific guidance as to how exactly to decompose** a given function into its sub-functions and we have to use subjective judgment to carry out decomposition.

Structured Design

- The **aim of structured design** is to **transform the results of the structured analysis** (i.e. a **DFD representation**) **into a structure chart**. Structured design provides **two strategies** to guide **transformation of a DFD into a structure chart**.
  1. **Transform analysis**
  2. **Transaction analysis**
- Starting with the level 1 DFD, **SD transforms DFD into module representation** **using either the transform or the transaction analysis** and then proceeds towards the lower-level DFDs.
- At each level of transformation, it is **important to** first **determine** whether the **transform or the transaction analysis** is **applicable** to a particular DFD.

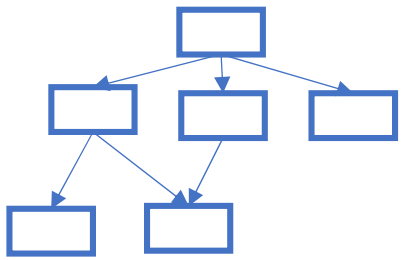
Structure Chart

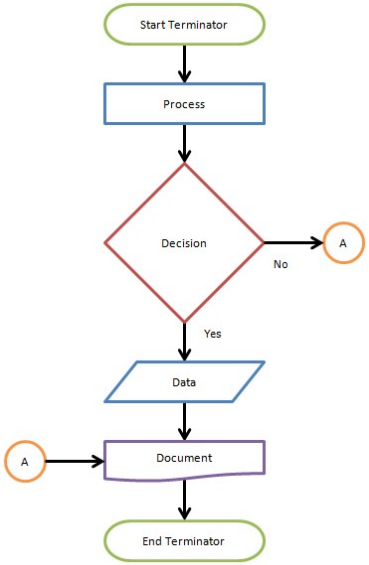


- A **structure chart** **represents** the **software architecture**
  - **various** **modules** making up the system
  - the **dependency** (which module calls which other modules), and
  - the **parameters that are passed** among the different modules
- The **structure chart** **representation** can be **easily** **implemented** using some **programming language**
- The basic building blocks which are used to design structure charts:
  - **Rectangular boxes:** Represents a **module**.
  - **Module invocation arrows:** **Control** is passed **from one module** to **another module** in the **direction** of the connecting arrow
  - **Data flow arrows: Arrows** are **annotated** with **data name**; named data passes from one module to another module in the direction of the arrow.
  - **Library modules:** Represented **by a rectangle with double edges**.
  - **Selection:** Represented by a **diamond symbol**.
  - **Repetition:** Represented by **a loop around the control flow arrow**.

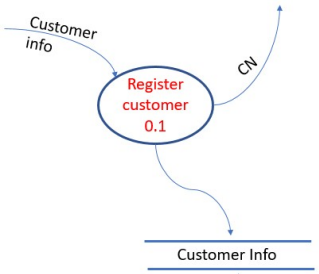
Structure Chart *vs*. Flow Chart

- **Flow chart** is a convenient technique **to represent the flow of control in a program**. A **structure chart differs** from a **flow chart** in three principal ways:
  - It is usually **difficult to identify the different modules** of the software **from its flow chart** representation.
  - **Data interchange among different modules is not represented in a flow chart**.
  - **Sequential ordering** of tasks inherent in a flow chart **is suppressed in a structure chart**.





Transform Analysis

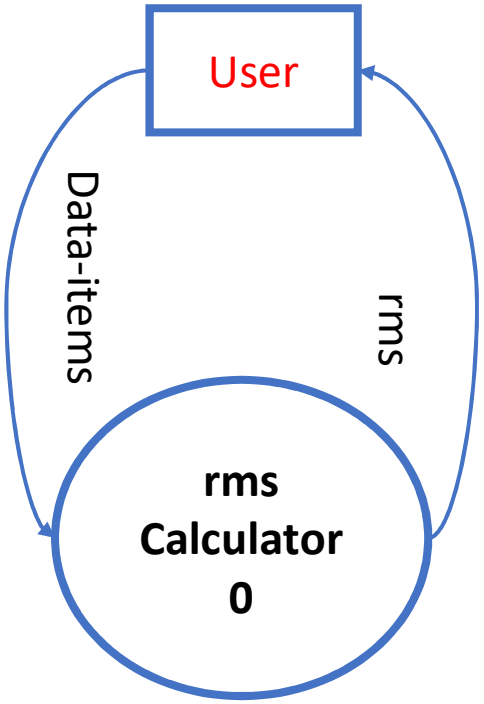


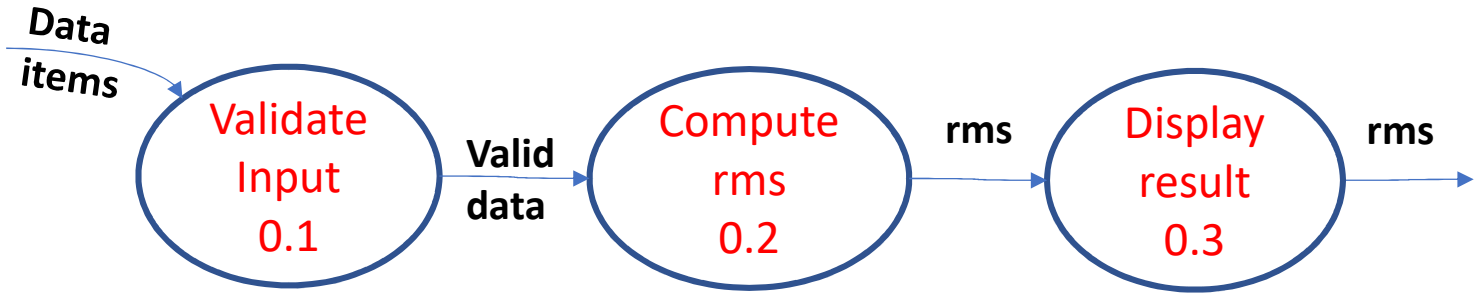
- **Transform analysis identifies** the primary **functional components (modules)** and the **high level inputs** and **outputs for these components**.
- **Step 1**: **Divide the DFD into 3 types of parts**:
  - **Input**: The **input portion of the DFD includes processes that transform input data from physical (e.g. character from terminal) to logical forms (e.g. internal tables, lists, etc.)**. Each input portion is called an **afferent branch** (branch that receives info)
  - **Output**: The **output portion of a DFD transforms output data from logical to physical form**. Each output portion is called an **efferent branch** (branch that dispatches info)
  - **Logical processing**: The **remaining portion of a DFD** related to logical processing is called the **central transform**.
- **Step 2**: **Derive the structure chart by drawing one functional component for the central transform, and the afferent and efferent branches. Draw these below a root module**, which would invoke these modules.

Transform Analysis

- **Step 3**:
  - **Refine the structure chart by adding sub-functions required** by each of the high-level functional components.
  - **Many levels of functional components may be added**.
  - This process of **breaking functional components into subcomponents is called factoring**.
  - Factoring may include addition of
    - **read and write modules,**
    - **error-handling modules,**
    - **initialization and termination process,**
  - The **factoring process is continued until all bubbles in the DFD are represented in the structure chart**.

RMS Calculating Software



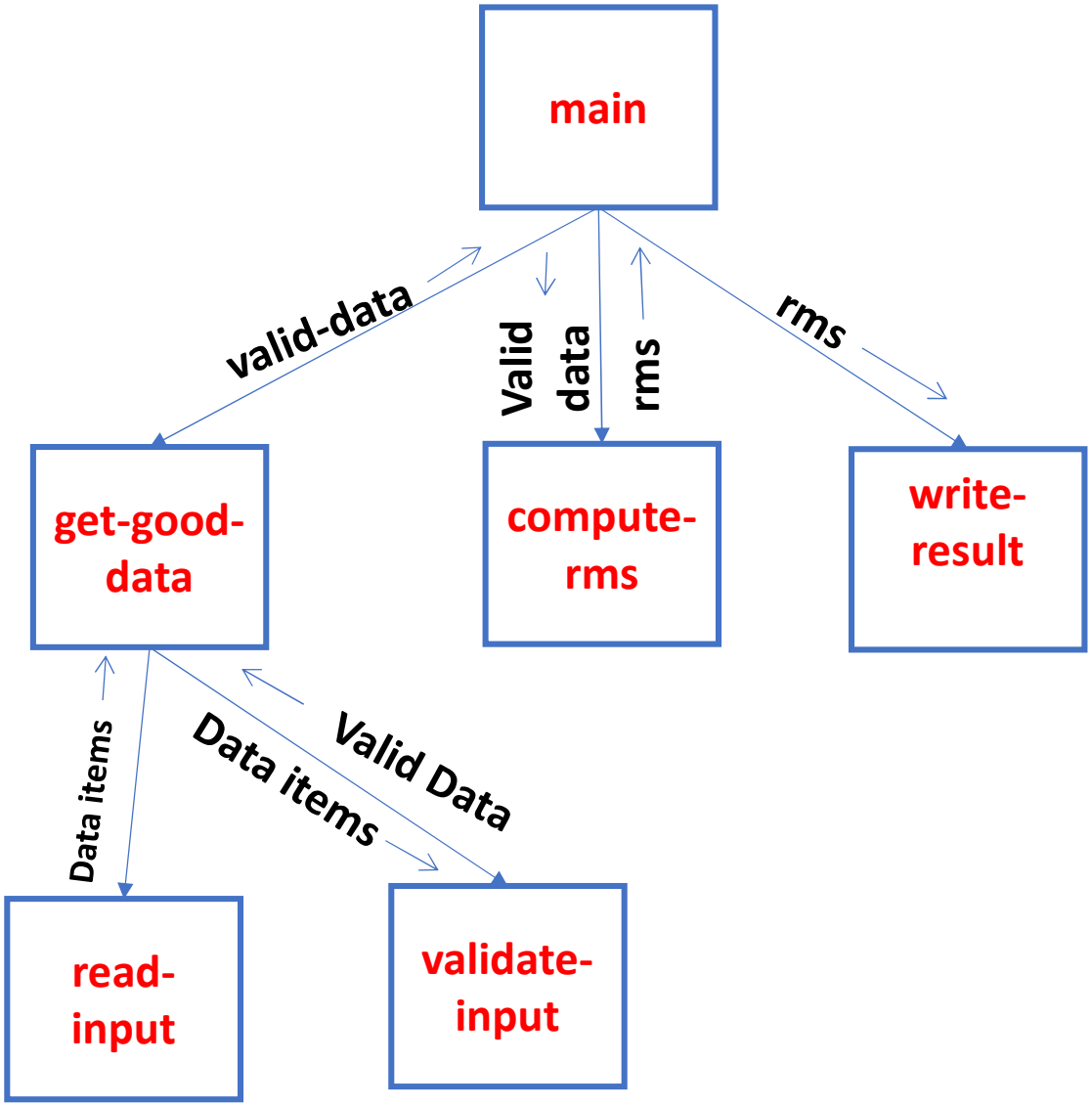


DFD level 0: Context Diagram

DFD level 1

Structure chart for RMS software

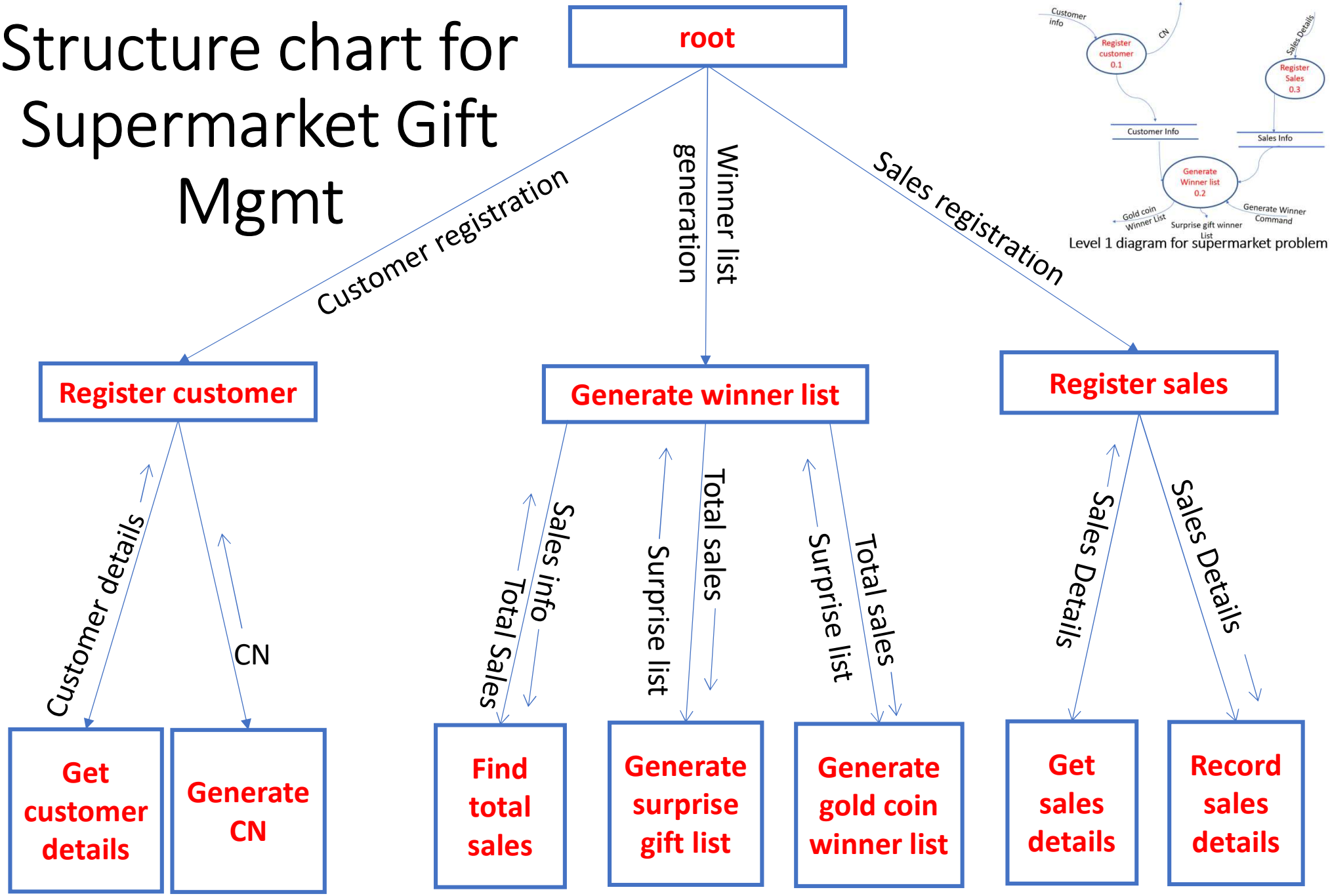
- From the level 1 DFD, it can be decided that
  - the **validate-input** can be considered as the **afferent** (**input**) **branch**
  - the **write-output** as the **efferent** (**output**) **branch** and
  - the remaining portion (i.e. **compute-rms**) forms the central transform
  - By applying the step 2 and step 3 of transform analysis, the structure chart can be prepared



Transaction Analysis

- A **transaction** is about some **meaningful piece of work done** by a user
- **Transaction analysis** is **useful for** **preparing structured chart for transaction processing programs**.
- In a **transaction-driven system, one of several possible paths through the DFD is traversed depending upon the input data item**.
  - **This is in contrast to a transform centered system** which is characterized by **similar processing steps for each data item**.
- **Each different way in which input data is handled is a transaction**. A simple way to identify a transaction is
  - **Check the input data**.
  - **The number of bubbles on which the input data to the DFD are incident defines the number of transactions**.
  - However, **some transaction may not require any input data**. These **transactions can be identified from the experience** of solving a large number of examples.

Structure chart for Supermarket Gift Mgmt



Level 1 diagram for supermarket problem

Transaction Analysis

- **For each identified transaction**
  - **Trace the input data to the output**.
  - All the **traversed bubbles belong to the transaction**. **These bubbles should be mapped to the same module on the structure chart**.
- In the **structure chart, draw a root module and below this module draw each identified transaction a module**.
- Every transaction carries a tag, which identifies its type.
- Transaction analysis uses this tag to divide the system into transaction modules and a transaction-center module.

Assignment -I

- A software system called **GenerateNumbers** that accepts values of two positive integers ***n*** and ***m*** and generate all **prime numbers** and all **Armstrong numbers** between *n* and *m*.
  - Prime number is a number that is divisible only by 1 and the same number, e.g. 19.
  - Armstrong number is a number whose value is equal to sum of the cubes of the digits of the number, e.g. 153 ($153=1^3+5^3+3^3$).
- Construct **hierarchies of DFDs** for the system, assuming **separate modules** for **accept input**, **validate input**, **generate prime numbers** and **Armstrong numbers**.

Assignment -II

- A library maintains information about the books and registered members. A member can **issue a book** for a certain period, **return** the issued book and also can **re-issue** a book. The library maintains the names, addresses, e-mail and phone no. of its member. At the time of **registration** the member provides personal details, pays annual membership fee and a fixed amount towards refundable caution deposit to become a member of the library for one year from day of registration. Each of these member assigned a unique member identification number (MIN) by the computer at the time of registration. There is an upper limit on maximum number of books a member can issue. If the number of books issued is less that the upper limit, then the member can **issue** a book for a month. Within a month of issue date, the member can either **return** the book or **re-issue** the book. Re-issue is allowed only once. If a book is not returned back within a month, there is a fine charged on per day basis. Member pays the fine at the time of returning the book. The member can **renew registration** by paying membership fee before expiry of membership period. A member can **claim clearance** from library at any time. Administration returns back caution deposit and give clearance if there is no book issued against the member. At the end of the year library administration **prepares a list of defaulter** members who are yet to return over due books and **send mail** for paying fine.
- Prepare a hierarchy of DFDs on this requirement.